



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
SEM 1 2025/2026
SECR3553 ARTIFICIAL INTELLIGENCE
(KEPINTARAN BUATAN)
SECTION 03

ASSIGNMENT 2:
PARKING DETECTION APPLICATION
THEME:
SMART CITY

Lecturer Name: Dr Norsham binti Idris

Group Members:

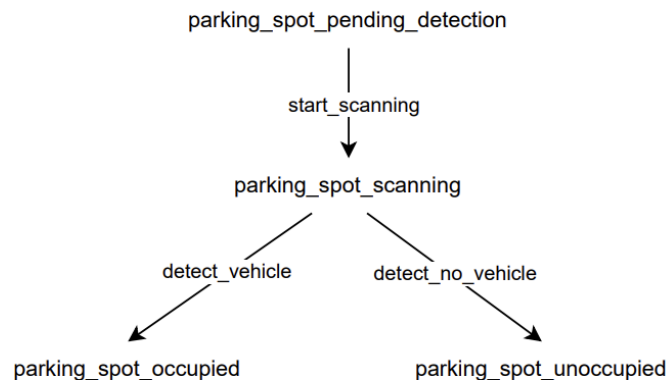
Name	Matric Number
NAWWARAH AUNI BINTI NAZRUDIN	A23CS0143
MUHAMMAD SYAHMI FARIS BIN RUSLI	A23CS0138
EVELYN GOH YUAN QI	A23CS0222
WOO CHENG SHUAN	A23CS0283
NG YU HIN	A23CS0148

TABLE OF CONTENT

1.0 Occupied Parking Space Detection	3
1.1 Explanation States and Actions	3
1.2 Problem Formulation and Explanation	4
2.0 Available Parking Space Detection	5
2.1 Explanation States and Actions	5
2.2 Problem Formulation and Explanation	6
3.0 Provide Nearest Available Parking Spot	7
3.1 Explanation States and Actions	7
3.2 Problem Formulation and Explanation	7
4.0 Update Parking Database	9
4.1 Explanation States and Actions	9
4.2 Problem Formulation and Explanation	10
5.0 Notify Driver When the Parking is Full	11
5.1 Explanation States and Actions	11
5.2 Problem Formulation and Explanation	12

Problem Formulation by State Space Graph

1.0 Occupied Parking Space Detection



1.1 Explanation States and Actions

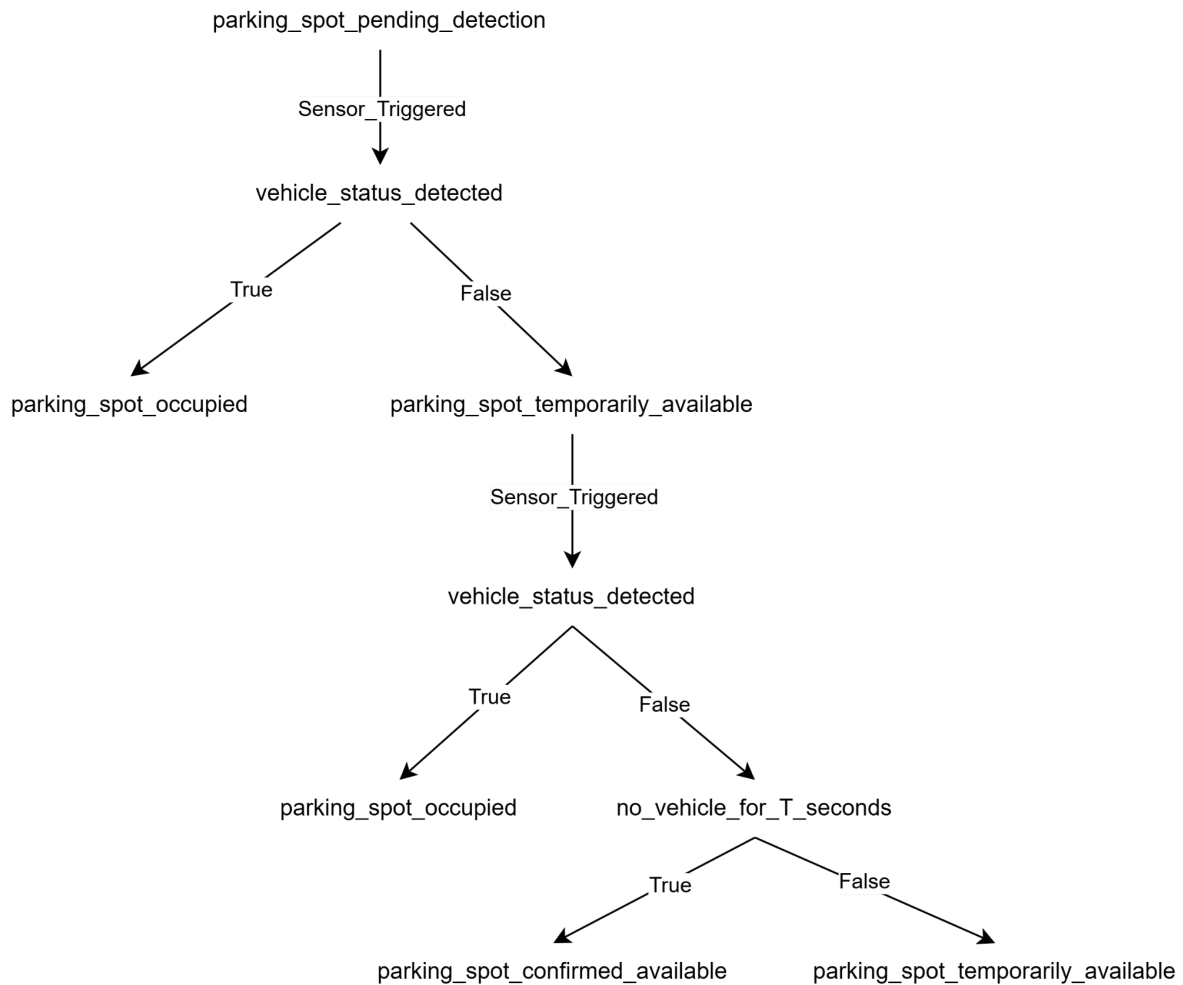
States	1. parking_spot_pending_detection: The system has not yet begun scanning this parking spot. Initial state. 2. parking_spot_scanning: Camera/sensor is actively scanning and AI is processing the image data to detect vehicle presence. Intermediate state. 3. parking_spot_occupied: AI has successfully detected a vehicle in the parking spot. The spot is confirmed occupied. Terminal state 4. parking_spot_unoccupied: AI has detected no vehicle is present in the parking spot. Terminal state
Actions	1. start_scanning: Initiates the detection process by activating cameras and beginning AI image analysis. 2. detect_vehicle: AI identifies a vehicle in the parking spot through computer vision algorithms. 3. detect_no_vehicle: AI detects the absence of any vehicle in the parking spot.

1.2 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	parking_spot_pending_detection	The system has not scanned the parking space yet. Image exists, but no detection has occurred.
Action	• start_scanning • detect_vehicle • detect_no_vehicle	The system performs one detection sequence: • start_scanning: Moves the system to the scanning phase. • detect_vehicle: If a car is detected, transition to parking_spot_occupied. • detect_no_vehicle: If no car is detected, transition to parking_spot_unoccupied.
Goal	parking_spot_occupied	The primary goal is to successfully detect occupied parking spaces
Path Cost	The total path cost is the number of actions taken from the initial state to the goal state in the detection process. Each action has a cost of 1 unit. Actions and their costs: • start_scanning = 1 • detect_vehicle / detect_no_vehicle = 1	The path cost measures the computational effort required to determine whether a parking spot is occupied. Each transition in the state space graph represents an operation performed by the AI system. • start_scanning (1 unit): The system begins analyzing the parking spot image. This includes loading the image and preparing it for detection. • detect_vehicle / detect_no_vehicle (1 unit): The AI performs the vehicle-detection process, which involves applying image recognition algorithms to check if a vehicle is present.

Solution: parking_spot_pending_detection, parking_spot_scanning, parking_spot_occupied

2.0 Available Parking Space Detection



2.1 Explanation States and Actions

States	parking_spot_pending_detection: The system has not yet checked this parking spot. Initial state. parking_spot_occupied: A vehicle is detected in the spot. Terminal state. parking_spot_temporarily_available: No vehicle detected yet. Waiting for continuous T seconds to confirm availability. Intermediate state. parking_spot_confirmed_available: No vehicle detected for a continuous period of T seconds. Terminal state; spot is officially available.
Actions	- True: action for state checking condition to be true - False: action for state checking condition to be false

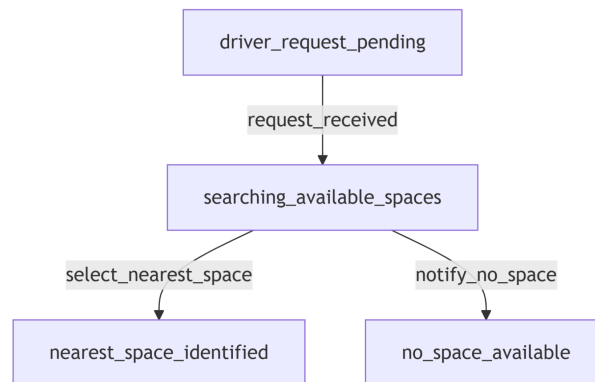
2.2 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	parking_spot_pending_detection	The system has not yet detected whether a vehicle is present. This is where the AI starts evaluating the parking spot.
Action	True / False checks: • vehicle_detected? • no_vehicle_for_T_seconds?	At each state, the system performs a True/False check to decide the next state: • If vehicle_detected = True, move to parking_spot_occupied. • If vehicle_detected = False, move to parking_spot_temporarily_available. • If no_vehicle_for_T_seconds = True, move to parking_spot_confirmed_available. • If no_vehicle_for_T_seconds = False, remain in temporarily available until T seconds pass.
Goal	Terminal states: • parking_spot_occupied • parking_spot_confirmed_available	These are the final outcomes the system aims to reach. Occupied represents a detected vehicle, Confirmed Available represents a verified empty spot
Path Cost	Number of sensor checks + waiting period (T seconds)	Each transition has a cost: • vehicle_detected or vehicle_not_detected = 1 unit per check • Waiting T seconds to confirm availability = T units The total path cost is the sum of all actions from pending_detection to a terminal state.

Solution:

parking_spot_pending_detection, parking_spot_temporarily_available,
parking_spot_confirmed_available

3.0 Provide Nearest Available Parking Spot



3.1 Explanation States and Actions

States	1. driver_request_pending: The driver has not made a request yet. Initial state. 2. searching_available_spaces: The system is checking the database/sensors for available parking spaces. Intermediate state. 3. nearest_space_identified: The system identifies and displays the nearest available parking space. Terminal state. 4. no_space_available: The system displays cannot find any available parking space. Terminal state.
Actions	1. request_received: The driver taps the “Find Parking” button. 2. select_nearest_space: The system calculates and chooses the closest available spot. 3. notify_no_space: The system informs the driver that no space is available.

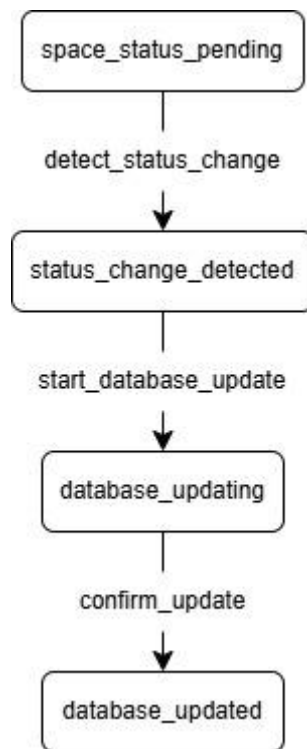
3.2 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	driver_request_pending	The system is waiting for the driver to request a parking spot. No action has been taken yet.
Action	• request_received • select_nearest_space • notify_no_space	The system executes a request process to locate the nearest available parking space: • request_received: The system

		receives a parking request and moves to searching_available_spaces. ● select_nearest_space: The system checks all available parking spaces, calculates the distance from the driver's location, and chooses the nearest available space. If a space is found, it moves to nearest_space_identified. ● notify_no_space: If the system cannot find any available parking space, it transitions to no_space_available.
Goal	nearest_space_identified	The goal is reached when the system either successfully identifies and displays the nearest available parking spot.
Path Cost	Goal reached: request_received + select_nearest_space = 2 units	The path cost shows the number of operations the system performs to complete the request. Each transition in the state diagram counts as a single unit: ● Request_received = 1 ● Select_nearest_space = 1 ● Notify_no_space = 1

Solution: driver_request_pending, searching_available_spaces, nearest_space_identified

4.0 Update Parking Database



4.1 Explanation States and Actions

States	space_status_pending: The system has identified a change in the parking space (either newly occupied or newly available), but the database has not yet been updated. status_change_detected: The system validates that a parking space has changed from Occupied to Available or from Available to Occupied. database_updating: The system is now writing the new parking status to the central database. database_updated: The database has been successfully updated with the parking status.
Actions	detect_status_change: The parking space has changed, according to the AI system. start_database_update: The system commences the process of writing new data to the database. confirm_update: The system confirms that the update has been correctly stored.

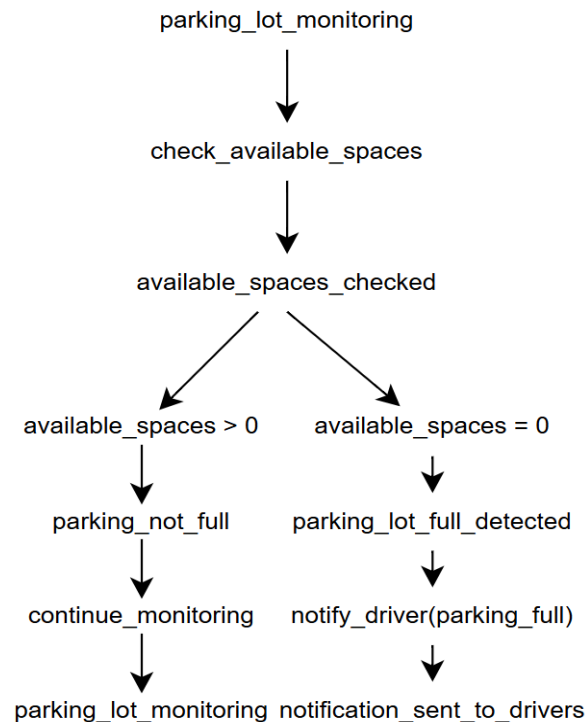
4.2 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	space_status_pending	The system recognizes that the status of a parking space has changed although it has not yet processed the update. This is the starting place after AI recognizes a new occupancy state.
Action	• detect_status_change • start_database_update • confirm_update	The sequence of action ensures the database reflects the correct parking status. • detect_status_change: AI concludes that a parking space's status has changed. • start_database_update: The new status is prepared and written into the database. • confirm_update: System guarantees the changed data is correctly stored and synchronized.
Goal	database_updated	This state-space formulation aims to guarantee that accurate, real-time parking status data is always present in the central database. This facilitates full-lot alerts, monitoring and driver direction.
Path Cost	Number of update actions	Each action costs 1 unit: • detect_status_change = 1 • start_database_update = 1 • confirm_update = 1 The total path cost represents the effort needed for the system to synchronize real-time data between the detection module and the parking application.

Solution:

space_status_pending, status_change_detected, database_updating, database_updated

5.0 Notify Driver When the Parking is Full



5.1 Explanation States and Actions

States	parking_lot_monitoring: The system is actively monitoring the number of available parking spaces in the parking lot. Initial state. available_spaces_checked: The system has completed checking the number of available spaces. Intermediate state. parking_not_full: The system detects that $\text{available_spaces} > 0$. The parking lot still has empty slots. Intermediate state. parking_lot_full: The system detects that $\text{available_spaces} = 0$. The parking lot is confirmed full. Terminal state. notification_sent_to_driver: The system has sent a notification informing drivers that the parking lot is full. Terminal state.
Actions	check_available_spaces: The system retrieves the number of available spaces from sensors or the database. continue_monitoring: If spaces are still available, the system loops back to continue observing the parking lot. notify_driver(parking_full): When the lot is full ($\text{available_spaces} = 0$), the system sends alerts to drivers.

5.2 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	parking_ot_monitoring	The system starts in a monitoring mode where it continuously observes the parking lot for updated space availability. No action has been taken yet.
Action	• check_available_spaces • continue_monitoring • notify_driver(parking_full)	The system performs a check to determine the number of available spaces: • If available_spaces>0, transition to parking_not_full, then repeat monitoring • If available_spaces=0, transition to parking_lot_full_detected and execute notify_driver(parking_full) to alert drivers.
Goal	notification_sent_to_drivers	The goal is achieved when the system successfully notifies drivers that the parking lot is full, preventing them from entering a full parking lot.
Path Cost	Number of checks + notification action	Each action has a cost of 1 unit: • check_available_spaces = 1 • continue_monitoring = 1 • notify_driver(parking_full) =1 The total path cost is the total number of monitoring cycles plus the cost of sending a notification once the lot becomes full.

Solution:

parking_lot_monitoring, available_spaces_detected, parking_lot_full_detected,
notification_sent_to_drives